

Отчёт по лабораторной работе №1

**Подготовка рабочего пространства. Модель
экспоненциального роста**

Глущенко Евгений Игоревич

2026-05-29

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Программная инженерия	7
3.2	Системы контроля версий	7
3.3	Модель экспоненциального роста	8
3.4	Литературное программирование	9
4	Выполнение лабораторной работы	10
4.1	Подготовка стенда	10
4.2	Структура проекта	11
4.3	Команды воспроизведения	11
4.4	Реализация модели	12
4.5	Базовый эксперимент	13
4.6	Параметрическое исследование	15
4.7	Генерация literate-материалов	17
5	Выводы	20
6	Приложение. Команды воспроизведения	21

Список иллюстраций

4.1	Переход в каталог проекта lab01	12
4.2	Активация окружения проекта через Pkg.instantiate()	12
4.3	Запуск базового сценария scripts/growth.jl	13
4.4	Экспоненциальный рост при $\alpha = 0.3$: численное решение и аналитическая кривая	14
4.5	Запуск сценария scripts/growth_parameters.jl	15
4.6	Траектории экспоненциального роста для разных α	16
4.7	Зависимость времени удвоения от α : имитация и теория $\ln 2/\alpha$	17
4.8	Запуск сценария scripts/tangle.jl	18
4.9	Проверка содержимого каталогов результатов	19

Список таблиц

4.1	Сводка базового эксперимента	14
4.2	Начало временного ряда growth_trajectory.csv	15
4.3	Результаты параметрического сканирования	16

1 Цель работы

Освоить инструменты программной инженерии (git, семантическое версионирование, общепринятые коммиты), подготовить рабочее пространство курса «Имитационное моделирование», познакомиться с языком Julia и пакетом DrWatson, реализовать модель экспоненциального роста в литературном стиле программирования, сгенерировать из единого источника производные форматы и провести параметрическое исследование модели.

Исполнитель работы: Глущенко Евгений Игоревич. Группа: НФИбд-01-23. Студенческий билет: 1132239110.

2 Задание

1. Создать рабочий каталог курса и рабочее пространство лабораторной работы.
2. Освоить семантическое версионирование и общепринятые коммиты.
3. Настроить git, ssh- и gpg-ключи.
4. Установить язык Julia и необходимые пакеты.
5. Выполнить предложенный код модели экспоненциального роста.
6. Преобразовать код в литературный стиль.
7. Сгенерировать из литературного кода чистый код, Jupyter notebook и документацию Quarto.
8. Выполнить код из notebook и интегрировать документацию в отчёт.
9. Добавить вычисление для набора параметров и повторить генерацию артефактов.

3 Теоретическое введение

3.1 Программная инженерия

Семантическое версионирование задаёт номер версии в виде МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ [1]: мажорная версия увеличивается при обратном несовместимых изменениях API, минорная — при добавлении функциональности с сохранением совместимости, патч — при обратном совместимых исправлениях.

Спецификация общепринятых коммитов (Conventional Commits) регламентирует структуру сообщений фиксации [2]. Сообщение имеет вид <тип>(<область>): <описание>, где основные типы — feat (новая функция, соответствует MINOR), fix (исправление, соответствует PATCH), а также docs, style, refactor, perf, test, build, ci. Критические изменения помечаются BREAKING CHANGE и соответствуют MAJOR.

3.2 Системы контроля версий

Git — распределённая система контроля версий, в которой каждый участник хранит полную копию истории. Базовые операции: `git init`, `git add`, `git commit`, `git push`, `git pull`, ветвление через `git checkout -b` и слияние `git merge --no-ff`. Для идентификации на сер-

верев репозиторийев применяются ssh-ключи (рекомендуется ed25519 или rsa 4096 бит), а для подтверждения авторства — подпись коммитов ключами gpg.

3.3 Модель экспоненциального роста

Экспоненциальный рост — процесс, в котором скорость роста величины пропорциональна её текущему значению [3]. Модель задаётся дифференциальным уравнением

$$\frac{du}{dt} = \alpha u, \quad u(0) = u_0,$$

где u — растущая величина (численность популяции, капитал), α — константа роста (мальтузианский параметр). Аналитическое решение

$$u(t) = u_0 e^{\alpha t}.$$

Время удвоения величины — время T_2 , за которое значение удваивается:

$$T_2 = \frac{\ln 2}{\alpha} \approx \frac{0,693}{\alpha}.$$

При $\alpha > 0$ наблюдается рост, при $\alpha < 0$ — экспоненциальное затухание. Модель применяется в биологии, финансах и в начальной фазе распространения инфекции [4].

3.4 Литературное программирование

Литературное программирование объединяет код и его описание в одном документе [5]. Проект организован на DrWatson.jl [6], а из единого литературного источника пакет Literate.jl [7] порождает чистый скрипт, документ Markdown и исполняемый Jupyter notebook. Расчёты выполнены на языке Julia [8].

4 Выполнение лабораторной работы

4.1 Подготовка стенда

Базовая настройка git — имя, почта, кодировка вывода и ветка по умолчанию:

```
git config --global user.name "Name Surname"  
git config --global user.email "work@mail"  
git config --global core.quotepath false  
git config --global init.defaultBranch master  
git config --global core.autocrlf input  
git config --global core.safecrlf warn
```

Генерация ssh-ключа для идентификации на сервере репозитория:

```
ssh-keygen -t ed25519 -C "Name Surname <work@mail>"
```

Генерация gpg-ключа для подписи коммитов и его экспорт:

```
gpg --full-generate-key
gpg --list-secret-keys --keyid-format LONG
gpg --armor --export <PGP Fingerprint>
```

4.2 Структура проекта

Рабочий каталог лабораторной работы организован в структуре DrWatson:

- `project/src/ExponentialGrowth.jl` — модуль с численным решением, аналитикой, временем удвоения, параметрическим сканированием и графиками;
- `project/scripts/growth.jl` — базовый эксперимент;
- `project/scripts/growth_parameters.jl` — параметрическое исследование;
- `project/scripts/growth_literate.jl`, `growth_parameters_literate.jl` — литературные источники;
- `project/scripts/tangle.jl` — генератор literate-артефактов;
- `project/data/`, `project/plots/`, `project/docs/`, `project/notebooks/` — таблицы, графики, markdown- и notebook-материалы;
- `report/` и `presentation/` — отчёт и презентация на Quarto.

4.3 Команды воспроизведения

```
cd ~/labs/lab01/project
~/juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
~/juliaup/bin/julia --project=. -e 'include("scripts/growth.jl")'
```

```
~/juliaup/bin/julia --project=. -e 'include("scripts/growth_parameters.jl")'
~/juliaup/bin/julia --project=. scripts/tangle.jl
```

4.3.1 Фактическое выполнение команд

Переход в каталог проекта (Рисунок 4.1) и активация зависимостей через `Pkg.instantiate()` (Рисунок 4.2).

```
lilidji@eiglushchenko:~$ cd ~/labs/lab01/project
```

Рисунок 4.1: Переход в каталог проекта lab01

```
lilidji@eiglushchenko:~/labs/lab01/project$ ~/.juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

Рисунок 4.2: Активация окружения проекта через `Pkg.instantiate()`

Окружение поднято из `Project.toml` и `Manifest.toml`, после чего все сценарии запускались в одном воспроизводимом окружении.

4.4 Реализация модели

Аналитическое решение и время удвоения заданы напрямую, а численное решение получено методом Рунге–Кутты 4-го порядка:

```
analytic_growth(u0, α, t) = u0 * exp(α * t)
doubling_time(α) = log(2) / α

function run_growth(u0, α, tspan; dt = 0.1)
    t0, tf = tspan
    nsteps = Int(round((tf - t0) / dt))
    u = Float64(u0)
```

```

# ... массивы ts, us, ua ...
for step in 1:nsteps
    k1 = α * u
    k2 = α * (u + dt / 2 * k1)
    k3 = α * (u + dt / 2 * k2)
    k4 = α * (u + dt * k3)
    u = u + dt / 6 * (k1 + 2k2 + 2k3 + k4)
    # запись ts, us, ua = analytic_growth(u0, α, t + dt)
end
return DataFrame(t = ts, u = us, u_analytic = ua, abs_error = abs.(us .
end

```

4.5 Базовый эксперимент

Для базового сценария использованы параметры $u_0 = 1.0$, $\alpha = 0.3$, интервал $t \in [0, 10]$, шаг $dt = 0.1$. Вывод запуска:

```

Exponential growth baseline completed
doubling time = 2.31

```

На Рисунок 4.3 показан фактический запуск базового сценария `scripts/growth.jl` с выводом сводной таблицы.

```

tllidji@eiglushchenko:~/labs/lab01/project$ ~/.juliaup/bin/julia --project=. -e 'include("scripts/growth.jl")'
Exponential growth baseline completed
doubling time = 2.31
1x6 DataFrame
  Row  u0      alpha  final_numeric  final_analytic  doubling_time  max_abs_error
  ---  ---  ---
  1    1.0    0.3      20.0855      20.0855      2.31049      3.96693e-7

```

Рисунок 4.3: Запуск базового сценария `scripts/growth.jl`

Сводка прогона приведена в Таблица 4.1.

Таблица 4.1: Сводка базового эксперимента

u_0	α	final_numerical	final_analytical	doubling_time	max_abs_error
1.0	0.3	20.0855	20.0855	2.3105	$3.97 \cdot 10^{-7}$

Численное решение совпадает с аналитическим $u(10) = e^3 \approx 20.0855$ с максимальной абсолютной погрешностью порядка 10^{-7} , что подтверждает корректность реализации метода Рунге-Кутты. График приведён на Рисунок 4.4.

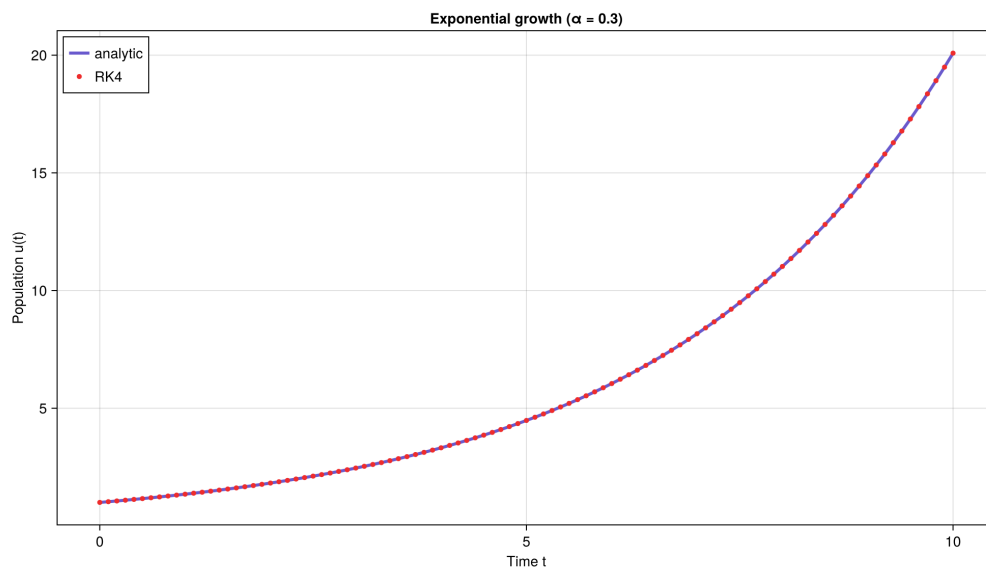


Рисунок 4.4: Экспоненциальный рост при $\alpha = 0.3$: численное решение и аналитическая кривая

Точки численного решения ложатся точно на аналитическую кривую $u(t) = u_0 e^{\alpha t}$.

4.5.1 Фрагмент временного ряда

`growth_trajectory.csv`

Первые строки временного ряда приведены в Таблица 4.2.

Таблица 4.2: Начало временного ряда growth_trajectory.csv

	t	u	u_analytic	abs_error
0.0	1.00000	1.00000	1.00000	0.0
0.1	1.03045	1.03045	1.03045	$2.0 \cdot 10^{-10}$
0.2	1.06184	1.06184	1.06184	$4.2 \cdot 10^{-10}$
0.3	1.09417	1.09417	1.09417	$6.5 \cdot 10^{-10}$
0.4	1.12750	1.12750	1.12750	$8.9 \cdot 10^{-10}$

4.6 Параметрическое исследование

В сценарии scripts/growth_parameters.jl исследуется влияние скорости роста на сетке $\alpha \in \{0.1, 0.3, 0.5, 0.8, 1.0\}$. Вывод сценария:

```
Exponential growth parameter scan completed
scenarios: 5
```

На Рисунок 4.5 показан фактический запуск параметрического исследования scripts/growth_parameters.jl.

```
llldji@eiglushchenko:~/labs/lab01/project$ ~/.juliaup/bin/julia --project=. -e 'include("scripts/growth_parameters.jl")'
Exponential growth parameter scan completed
scenarios: 5
5x5 DataFrame
 Row   alpha   final_numeric   final_analytic   doubling_time   max_abs_error
  Float64   Float64       Float64         Float64        Float64
1     0.1      2.71828         2.71828         6.93147        2.24641e-10
2     0.3     20.0855         20.0855         2.31049        3.96693e-7
3     0.5     148.413         148.413         1.38629        3.70729e-5
4     0.8     2980.95         2980.96         0.866434       0.00761551
5     1.0    22026.3         22026.5         0.693147       0.168894
```

Рисунок 4.5: Запуск сценария scripts/growth_parameters.jl

Результаты приведены в Таблица 4.3.

Таблица 4.3: Результаты параметрического сканирования

α	final_numeric	final_analytic	doubling_time	max_abs_error
0.1	2.7183	2.7183	6.9315	$2.2 \cdot 10^{-10}$
0.3	20.0855	20.0855	2.3105	$4.0 \cdot 10^{-7}$
0.5	148.413	148.413	1.3863	$3.7 \cdot 10^{-5}$
0.8	2980.95	2980.96	0.8664	$7.6 \cdot 10^{-3}$
1.0	22026.3	22026.5	0.6931	$1.7 \cdot 10^{-1}$

С ростом α итоговая численность растёт экспоненциально (от $e^1 \approx 2.72$ до $e^{1.0} \approx 22026$), а время удвоения убывает по закону $T_2 = \ln 2 / \alpha$. Абсолютная погрешность численного решения растёт вместе с величиной u , но относительная погрешность остаётся малой (порядка 10^{-6} даже при $\alpha = 1.0$). Сравнение траекторий приведено на Рисунок 4.6.

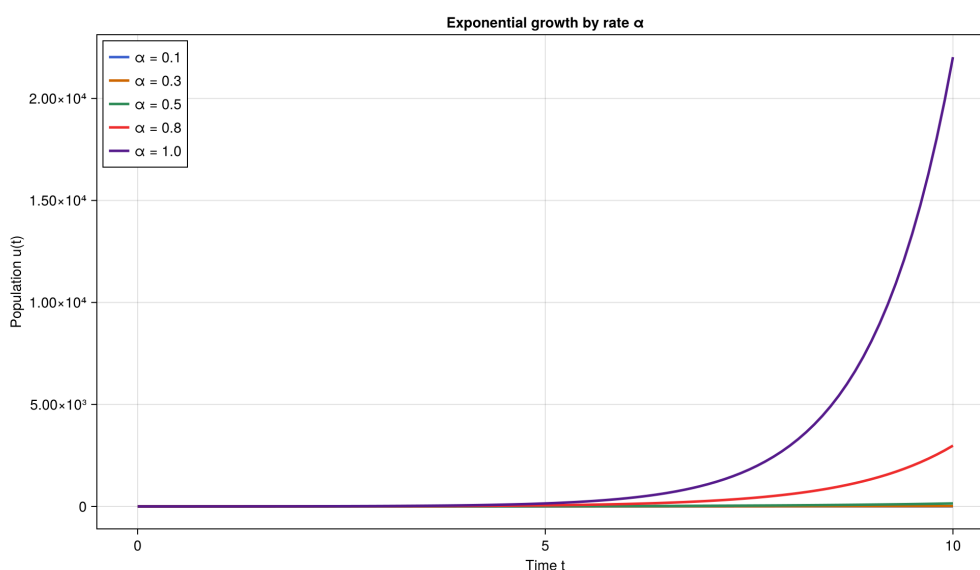


Рисунок 4.6: Траектории экспоненциального роста для разных α

Чем больше α , тем круче кривая роста. На Рисунок 4.7 показана

зависимость времени удвоения от α .

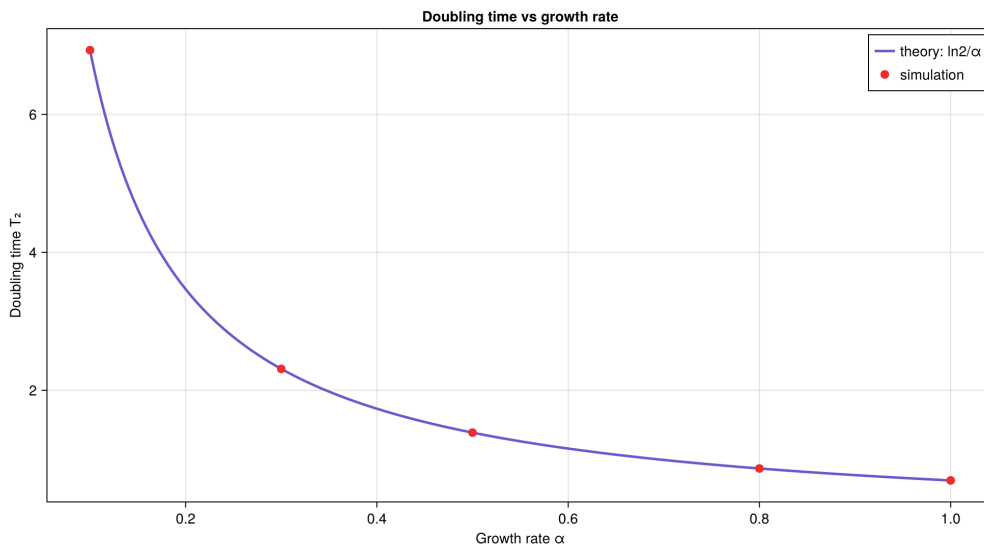


Рисунок 4.7: Зависимость времени удвоения от α : имитация и теория $\ln 2/\alpha$

Имитационные точки точно ложатся на теоретическую гиперболу $T_2 = \ln 2/\alpha$, что подтверждает согласованность численного решения с аналитикой.

4.7 Генерация `iterate`-материалов

Сценарий `scripts/tangle.jl` формирует производные представления из литературных источников:

```
for script_path in scripts
    name = replace(splitext(basename(script_path))[1], "_iterate" => "")
    Literate.script(script_path, scriptsdir(); name = "$(name)_clean", credit = " ")
    Literate.notebook(script_path, projectdir("notebooks"); name = name, credit = " ")
    Literate.markdown(script_path, projectdir("docs"); name = name, credit = " ")
end
```

После запуска были получены docs/growth.md, docs/growth_parameters.md, notebooks/growth.ipynb, notebooks/growth_parameters.ipynb, scripts/growth_clean.jl, scripts/growth_parameters_clean.jl. Вывод сценария:

```
generated for growth
generated for growth_parameters
```

На Рисунок 4.8 показан фактический запуск scripts/tangle.jl с генерацией clean-скриптов, markdown-документов и исполняемых notebook-файлов.



```
lilidji@eiglushchenko:~/labs/lab01/project$ ~/.juliaup/bin/julia --project=. scripts/tangle.jl
[ Info: generating plain script file from `~/labs/lab01/project/scripts/growth_literate.jl`
[ Info: writing result to `~/labs/lab01/project/scripts/growth_clean.jl`
[ Info: generating notebook from `~/labs/lab01/project/scripts/growth_literate.jl`
[ Info: executing notebook `growth.ipynb`
[ Info: writing result to `~/labs/lab01/project/notebooks/growth.ipynb`
[ Info: generating markdown page from `~/labs/lab01/project/scripts/growth_literate.jl`
[ Info: writing result to `~/labs/lab01/project/docs/growth.md`
generated for growth
[ Info: generating plain script file from `~/labs/lab01/project/scripts/growth_parameters_literate.jl`
[ Info: writing result to `~/labs/lab01/project/scripts/growth_parameters_clean.jl`
[ Info: generating notebook from `~/labs/lab01/project/scripts/growth_parameters_literate.jl`
[ Info: executing notebook `growth_parameters.ipynb`
[ Info: writing result to `~/labs/lab01/project/notebooks/growth_parameters.ipynb`
[ Info: generating markdown page from `~/labs/lab01/project/scripts/growth_parameters_literate.jl`
[ Info: writing result to `~/labs/lab01/project/docs/growth_parameters.md`
generated for growth_parameters
```

Рисунок 4.8: Запуск сценария scripts/tangle.jl

Notebook-файлы выполняются при генерации (execute = true), что подтверждает работоспособность кода. После генерации выполнена проверка состава каталогов data/, plots/, docs/ и notebooks/ (Рисунок 4.9).

```
lilidji@eiglushchenko:~/labs/lab01/project$ ls -1 data
ls -1 plots
ls -1 docs
ls -1 notebooks
growth_parameter_scan.csv
growth_summary.csv
growth_trajectory.csv
growth_base.png
growth_comparison.png
growth_doubling.png
growth.md
growth_parameters.md
growth.ipynb
growth_parameters.ipynb
lilidji@eiglushchenko:~/labs/lab01/project$
```

Рисунок 4.9: Проверка содержимого каталогов результатов

5 Выводы

В ходе работы освоены инструменты программной инженерии (семантическое версионирование, общепринятые коммиты, git, ssh, gpg) и подготовлено рабочее пространство курса. Реализована модель экспоненциального роста $du/dt = \alpha \cdot u$ методом Рунге-Кутты 4-го порядка; численное решение совпало с аналитическим $u(t) = u_0 e^{\alpha t}$ с погрешностью порядка 10^{-7} для базового сценария. Параметрическое исследование на сетке α подтвердило закон убывания времени удвоения $T_2 = \ln 2 / \alpha$. Код оформлен в литературном стиле, из единого источника с помощью `Literate.jl` получены чистые скрипты, markdown-документы и исполняемые notebook-файлы.

6 Приложение. Команды воспроизведения

```
cd ~/labs/lab01/project
~/juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
~/juliaup/bin/julia --project=. -e 'include("scripts/growth.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/growth_parameters.jl")'
~/juliaup/bin/julia --project=. scripts/tangle.jl
cd ~/labs/lab01/report
quarto render simulation-modeling-lab01-report.qmd --to pdf
quarto render simulation-modeling-lab01-report.qmd --to docx
cd ~/labs/lab01/presentation
quarto render simulation-modeling-lab01-presentation.qmd --to beamer
quarto render simulation-modeling-lab01-presentation.qmd --to revealjs
quarto render simulation-modeling-lab01-presentation.qmd --to pptx
```

1. Preston-Werner T. Semantic Versioning 2.0.0. 2013.
2. Conventional Commits. Conventional Commits 1.0.0. 2023.
3. Malthus T. R. An Essay on the Principle of Population. London: J. Johnson, 1798.
4. Strogatz S. H. Nonlinear Dynamics and Chaos. 2-е изд. Boulder, CO: Westview Press, 2014.

5. Knuth D. E. Literate Programming // The Computer Journal. 1984. Т. 27, № 2. С. 97–111.
6. Datseris G. и др. DrWatson.jl Documentation. 2024.
7. Ekre F., contributors. Literate.jl Documentation. 2024.
8. Bezanson J. и др. Julia: A Fresh Approach to Numerical Computing. 2017.